Differentially Private Bipartite Graph Synthesis

Quan Yuan
Hong Kong Baptist University
quanyuan@comp.hkbu.edu.hk

Jianliang Xu
Hong Kong Baptist University
xujl@comp.hkbu.edu.hk

ABSTRACT

Bipartite graphs, which consist of two disjoint vertex sets, are ubiquitous in real-world applications, such as user-item purchase records and user-webpage browsing records. However, directly releasing real bipartite graph data can pose serious privacy risks because these data contain sensitive information. Differential privacy (DP) provides quantifiable privacy guarantees and has been widely adopted for graph protection. Nevertheless, existing DP graph synthesis approaches mainly focus on general graph protection and overlook the distinctive property of bipartite graphs, resulting in limited utility for bipartite graph release. In particular, publishing a bipartite graph under DP is challenging because excessive noise injection or limited information extraction can lead to a low-quality reconstructed graph. To limit the impact of substantial noise, we exploit the vertex degrees on both sides of the bipartite graph to obtain a compact representation. For settings in which the impact of noise is insignificant, we design a block partitioning mechanism to capture finer-grained information. To accommodate different noise levels and data distributions, we further propose an adaptive perturbation strategy that dynamically determines whether to partition a block based on the privacy budget and the information already perturbed. We also develop a series of post-processing techniques for use during edge generation to improve the utility of synthetic bipartite graphs. We conduct extensive experiments on multiple real-world bipartite graph datasets. The experimental results demonstrate that our proposed methods outperform the evaluated baselines in the tested settings.

PVLDB Reference Format:
Quan Yuan and Jianliang Xu. Differentially Private Bipartite Graph Synthesis. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/qy115/DP-BGS.

1 INTRODUCTION

Bipartite graphs, whose vertices can be divided into two disjoint independent sets, are prevalent in real-world applications such as user-item networks in e-commerce [25] and user-page networks in social analysis [76]. Analyzing these data supports services such as including task assignment [56], behavior analysis [42], and advertising recommendations [68]. For example, link prediction can be conducted through the analysis of bipartite graphs [52]. However, bipartite graph data often contain sensitive personal information, such as click histories and purchase records, hindering the sharing of these data [38].

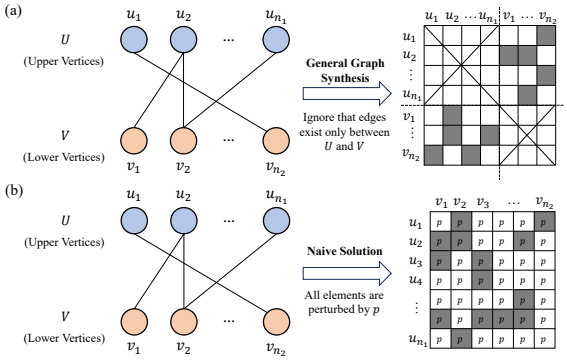


Figure 1: An example of a bipartite graph (left side) and the approaches used by general graph synthesis and the naive solution (right side).

Differential privacy (DP) [10] has been widely used to protect sensitive information in graph data [22, 29, 44, 61, 63, 67]. The core idea of DP in graph research is to ensure that the presence or absence of a single node or edge has a limited influence on the final output [24]. Many previous studies on differentially private graph analysis develop specialized methods for specific tasks, such as degree distribution [16] and subgraph counting [8, 54]. However, these task-specific approaches are not sufficiently general to support arbitrary downstream tasks, which limits their applicability.

In this paper, we focus on differentially private bipartite graph release to support various graph analysis tasks. Several studies have investigated *general graph synthesis* with DP guarantees [7, 36, 63, 69]. These approaches either encode the original graph into a specific representation, perturb that representation, and reconstruct a synthetic graph, or design mechanisms that perturb the adjacency matrix directly. The most straightforward approach to *bipartite graph synthesis* is to treat a bipartite graph as a general graph and directly apply these existing methods to generate a synthetic graph, as shown in Figure 1a. However, such approaches do not fully exploit the distinctive property of bipartite graphs, namely that edges exist only between the two vertex sets and not within either set. As a result, the generated graphs may fail to preserve key characteristics of the original bipartite graph. Another intuitive approach is to independently perturb each entry of the bipartite adjacency matrix with a certain probability p, as shown in Figure 1b. However, this randomized perturbation introduces substantial noise, which can distort the graph structure and lead to severe overestimation of many graph statistics. The limitations of existing methods give rise to the following three key challenges.

Challenge 1: How can we reduce the impact of noise and accurately reconstruct the graph when the perturbation is substantial? Under

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

a limited privacy budget, the perturbation introduces substantial noise, so the perturbation targets must be chosen carefully. Reconstructing a bipartite graph that closely matches those perturbed targets is also challenging. To address this challenge, we propose DBGS, a degree-based bipartite graph synthesis method. The key idea is to characterize the bipartite graph by the degrees of vertices in both parts. Vertex degrees provide a compact representation of the graph structure and reduce the dimensionality of the data that must be perturbed. We then design a sequential optimization procedure that aligns the degree sequences of the upper and lower vertex sets in the synthesized graph with the perturbed degree sequences.

Challenge 2: How can we extract and restore more useful structural information when the impact of noise is small? The DBGS method described before limits the impact of noise under a tight privacy budget. However, it becomes less suitable when the privacy budget is larger or the graph contains many high-degree vertices, because vertex degrees capture only limited structural information. Two bipartite graphs can have similar degree distributions while exhibiting substantially different connectivity patterns. To address this challenge, we further propose BKGS, a block-based bipartite graph synthesis method. In addition to using the vertex degrees, BKGS partitions each vertex set into two groups according to the perturbed degree information, producing a 2×2 block structure. It then collects degree statistics within each block to capture finer-grained structural information. We also design an edge-consistency procedure that enforces consistency between the global degree sequences and the block-level degree sequences, improving the utility of the synthesized bipartite graph.

Challenge 3: How can we maintain strong synthesis performance across different noise levels and data distributions? DBGS and BKGS are effective in specific scenarios, but their fixed perturbation strategies make it difficult to perform well across diverse settings. We therefore propose JBGS, a judgment-based bipartite graph synthesis method that selects the perturbation strategy according to the graph feature and the available privacy budget. Its key idea is to use the information already perturbed and the remaining privacy budget to compare the estimated error from additional noise with the estimated information gain from further partitioning. If the estimated noise error dominates the potential information gain, the current block is not partitioned further. Otherwise, the high-degree block is partitioned, and the same test is applied recursively to its sub-blocks. This design balances perturbation error and information extraction across different settings.

Contributions. The main contributions of this paper are summarized as follows.

- To the best of our knowledge, this is the first systematic study of differentially private bipartite graph synthesis.
- For noise-dominated settings, we introduce DBGS to limit the impact of substantial noise by using the degree sequences of the upper and lower vertex sets as a compact representation. We further design a sequential optimization method that restores degree information in both parts.
- For low-noise settings, we design BKGS to partition the bipartite graph into finer-grained blocks, enabling the extraction of more useful structural information.
- To accommodate different noise levels and data distributions, we develop JBGS to select the perturbation strategy according to the characteristics of the graph and the available privacy budget.
- We evaluate the proposed methods on multiple datasets using several metrics, and the results demonstrate their effectiveness.

Roadmap. The rest of this paper is organized as follows. In Section 2, we present background on DP. Next, we formally define the differentially private bipartite graph synthesis problem and introduce a naive solution in Section 3. We describe our methods in Section 4. In Section 5, we evaluate our methods through experiments on multiple real-world bipartite graph datasets. We discuss related work in Section 6 and conclude the paper in Section 7.

2 PRELIMINARIES

2.1 Differential Privacy

Differential privacy [10] was designed for privacy protection where a trusted data curator collects data from individual users, perturbs the aggregated results, and then publishes them.

Definition 1 (ε-Differential Privacy). *An algorithm $\mathcal{M}$ satisfies ε-differential privacy (ε-DP), where $\varepsilon > 0$, if and only if for any two neighboring datasets D and D', and for any possible output set O, we have*

$$\Pr[\mathcal{M}(D) \in O] \leq e^{\varepsilon} \Pr[\mathcal{M}(D') \in O].$$

Laplace Mechanism. This mechanism achieves DP guarantees by adding random Laplace noise to the aggregated results. The magnitude of the added noise depends on GS_f, *i.e.*, the *global sensitivity*,

$$GS_f = \max_{D \simeq D'} \| f(D) - f(D') \|_1,$$

where f is the aggregation function. When f outputs a scalar, the Laplace mechanism $\mathcal{M}$ is defined as follows:

$$\mathcal{M}_f(D) = f(D) + Lap\left(\frac{GS_f}{\varepsilon}\right),$$

where $Lap(\beta)$ denotes a random variable sampled from the Laplace distribution $\Pr[Lap(\beta) = x] = \frac{1}{2\beta} e^{-|x|/\beta}$.

Randomized Response Mechanism. The randomized response (RR) mechanism can be used to protect the privacy of binary variables [57]. Given a privacy budget ε, the probability p of outputting a true binary variable is as follows:

$$p = \frac{e^{\varepsilon}}{e^{\varepsilon} + 1}.$$

Composition Properties of DP. The following properties are commonly used for building complex differentially private algorithms from simpler subroutines [11].

- **Sequential Composition.** Combining m subroutines, where subroutine i satisfies that satisfy ε_i-DP, yields a mechanism satisfying ε-DP for $\varepsilon = \sum_i \varepsilon_i$.
- **Parallel Composition.** Given m algorithms operating on disjoint subsets of the data, with algorithm i satisfying ε_i-DP, the combined output satisfies ε-DP for $\varepsilon = \max\{\varepsilon_i\}$.
- **Post-processing.** Given an algorithm $\mathcal{M}$ satisfying ε-DP, releasing $g(\mathcal{M}(D))$ for any post-processing function g that does not access the private data beyond $\mathcal{M}(D)$ still satisfies ε-DP. In

other words, post-processing an output of a differentially private algorithm does not lead to additional privacy loss.

2.2 Differential Privacy for Bipartite Graphs

In this work, we consider an undirected and unweighted bipartite graph, denoted by $G = (U, V, E)$, where U and V are two disjoint vertex sets and $E \subseteq U \times V$ is the edge set. For ease of presentation, we refer to U and V as the upper and lower vertex sets, respectively. Let $n_1 = |U|$ and $n_2 = |V|$ denote the numbers of vertices in the upper and lower vertex sets, respectively, and let $m = |E|$ denote the number of edges in the bipartite graph. The corresponding adjacency matrix $\mathcal{A}$ has dimensions $n_1 \times n_2$. For any $u \in U$ and $v \in V$, $\mathcal{A}[u, v] = 1$ if there exists an edge between u and v, and $\mathcal{A}[u, v] = 0$ otherwise.

The definition of neighboring datasets is the basis for differential privacy, determining the target and strength of the privacy protection. In common differential privacy, two datasets are considered neighboring if they differ in a single individual's record. Applying DP to bipartite graph analysis therefore requires a suitable notion of neighboring graphs.

Definition 2 (Edge-Neighboring Bipartite Graphs). *Given two bipartite graphs $G = (U, V, E)$ and $G' = (U, V, E')$ with the same vertex sets U and V, G and G' are edge neighboring graphs if their edge sets differ by exactly one edge, i.e., $|E \triangle E'| = 1$, where $E \triangle E'$ denotes the symmetric difference between E and E'.*

Here, we regard the vertex information as public and focus on edge-level privacy for bipartite graphs. This setting corresponds to practical scenarios where the existence of a specific interaction needs to be protected, such as whether a user purchases a particular product or clicks on a particular website. This is reasonable because the identities of users or products may not be sensitive by themselves, while the interaction between them can reveal sensitive information about a user's preferences or behavior.

Definition 3 (ε-Edge Differential Privacy). *An algorithm $\mathcal{M}$ satisfies ε-edge differential privacy (ε-edge DP), where $\varepsilon > 0$, if and only if for any two edge neighboring bipartite graphs G and G', and for any possible output set O, we have*

$$\Pr[\mathcal{M}(G) \in O] \leq e^{\varepsilon} \Pr[\mathcal{M}(G') \in O].$$

3 PROBLEM DEFINITION

We consider the following problem: *Given a bipartite graph G, how can we generate, under edge DP, a synthetic bipartite graph G_s with properties similar to those of G?* By the post-processing property of DP, the synthetic bipartite graph G_s can be used for downstream graph analysis without additional privacy loss.

For differentially private bipartite graph synthesis, a naive solution is to independently perturb each entry of the bipartite adjacency matrix using the randomized response mechanism, as illustrated in Algorithm 1. Given the privacy budget ε, Naive retains the original value of each adjacency matrix entry with probability $p_e = \frac{e^{\varepsilon}}{1+e^{\varepsilon}}$, and flips the value with probability $1 - p_e$. The perturbed entries form the adjacency matrix of a synthetic bipartite graph G_s.

However, this method can introduce substantial distortion to the graph structure. For sparse bipartite graphs, the number of zero entries in the adjacency matrix is much larger than the number of one entries due to its sparse nature. Consequently, randomized response can convert a large number of zero entries into one entries, resulting in a synthetic graph that is considerably denser than the original graph. This density inflation can further lead to severe overestimation of various graph statistics and significantly degrade the utility of the synthetic bipartite graph.

Algorithm 1: Naive

Input: Original graph $G = (U, V, E)$, privacy budget ε.
Output: Synthetic bipartite graph G_s

```
p_e = e^ε / (1+e^ε)
for u ∈ U do
    for v ∈ V do
        if Random(0, 1) < p_e then
            A_s[u, v] = A[u, v]
        else
            A_s[u, v] = 1 − A[u, v]
G_s ← the noisy graph from A_s
```

4 METHODOLOGY

We introduce the three proposed graph synthesis methods in Section 4.1 through Section 4.3. We then summarize these methods in Section 4.4. In Section 4.5, we analyze the privacy guarantee, utility, and computational cost of the proposed algorithms.

4.1 Degree-based Graph Synthesis (DBGS)

To address the substantial noise introduced by the Naive solution, we design a degree-based bipartite graph synthesis method DBGS, as shown in Algorithm 2.

Overview. Instead of perturbing every entry of the adjacency matrix, DBGS perturbs the degrees of vertices on both sides of the bipartite graph. This strategy preserves important feature information while substantially limiting the impact of noise.

After obtaining the perturbed degrees on both sides, we face another key challenge: reconstructing a bipartite graph whose degree sequences closely match these values. Reconstructing the graph based solely on one side's degree sequence can lead to substantial discrepancies on the other side, whereas jointly optimizing the degree constraints of both sides may incur considerable computational overhead. We therefore adopt a sequential optimization strategy. Specifically, we first construct an initial graph based on the degree constraints of the upper vertices, and then adjust the edges according to the degree constraints of the lower vertices while keeping the degrees of the upper vertices unchanged. The detailed procedure of DBGS is as follows:

Degree Perturbation. In the first stage (shown in the DegPert function), we extract the degree sequences of the upper and lower vertex sets. To satisfy DP, Laplace noise is independently added to these degree values, followed by a post-processing step to ensure that all released degrees are valid integers. Since the addition or removal of a single edge changes the degree of exactly one vertex in each vertex set by one, the sensitivity of each vertex set's degree sequence is 1. Compared with Naive, DBGS substantially reduces the dimensionality of the information that needs to be perturbed.

Consequently, the released degree sequences provide a more compact representation of the original bipartite graph and can help preserve the key feature under noise.

Degree Restoration. As introduced in the DegRe function, we first construct an initial bipartite graph according to the perturbed degrees of the upper vertices. For each upper vertex, we randomly select distinct lower vertices and connect them to it until its perturbed degree constraint is satisfied. This step ensures that the initial graph matches the perturbed degree sequence of the upper vertices and provides a starting point for subsequent refinement. Next, we calculate the difference between the perturbed lower degree sequence $\hat{d}_V$ and the lower degree sequence d_{Vs} of the initial graph G_s. Based on the signs of these differences Δd_V, we assign lower vertices with nonzero differences to two sets, S_1 and S_2. Specifically, S_1 contains vertices that require additional edges ($\Delta d_V > 0$), whereas S_2 contains vertices whose current degrees exceed their perturbed targets ($\Delta d_V < 0$).

We then traverse all upper vertices to refine the lower vertex degree sequence. For each upper vertex u, let d_{u1} and d_{u2} denote the numbers of edges from u to S_1 and S_2, respectively. Then, we determine the number of edges that can be relocated from S_2 to S_1 for vertex u as $s_u = \min(|S_1| - d_{u1}, d_{u2})$. Here, s_u represents the maximum number of edges that can be relocated for vertex u under the current configuration. If $s_u > 0$, we randomly select s_u vertices from S_1 that are not currently connected to u, and add edges between u and these vertices. We then remove s_u existing edges between u and randomly selected vertices in S_2. Since each edge removed from S_2 is replaced by an edge to S_1, the degree of u remains unchanged. Meanwhile, the degrees of vertices in S_1 increase, while those in S_2 decrease, thereby reducing the discrepancy between the synthesized and perturbed degree sequences of the lower vertices.

A single traversal may not achieve a satisfactory match between the synthesized and perturbed degree sequences. We therefore repeat the correction process for at most R rounds. The process terminates when either R rounds have been completed or no edges are relocated during an entire traversal. This sequential optimization strategy avoids the computational overhead of jointly optimizing the degree constraints on both sides while effectively refining the degree sequence of the lower vertices without altering that of the upper vertices.

Algorithm 2: Degree-based graph synthesis (DBGS)

Input: Original graph $G = (U, V, E)$, privacy budget ε, the budget ratio for upper (lower) vertices r_u ($r_v = 1 - r_u$), maximum number of rounds R.
Output: Synthetic bipartite graph G_s

Step 1: Degree Perturbation
$\hat{d}_U, \hat{d}_V \leftarrow \text{DegPert}(G, \varepsilon, r_u)$
Step 2: Degree Restoration
$G_s \leftarrow \text{DegRe}(U, V, \hat{d}_U, \hat{d}_V, R)$

Function `DegPert`(G, ε, r_u)**:**
  $d_U, d_V \leftarrow G$ // Obtain the true degree sequences
  $\varepsilon_U \leftarrow r_u \cdot \varepsilon, \varepsilon_V \leftarrow (1 - r_u) \cdot \varepsilon$ // Allocate privacy budget
  $\tilde{d}_U \leftarrow d_U + Lap(\frac{1}{\varepsilon_U}), \tilde{d}_V \leftarrow d_V + Lap(\frac{1}{\varepsilon_V})$
  $\hat{d}_U \leftarrow \max(0, round(\tilde{d}_U)), \hat{d}_V \leftarrow \max(0, round(\tilde{d}_V))$
  return $\hat{d}_U, \hat{d}_V$;

Function `DegRe`$(U, V, \hat{d}_U, \hat{d}_V, R)$**:**
  $G_s \leftarrow NULL, i \leftarrow 0, flag \leftarrow 1$
  // Edge initialization: upper vertex degree constraint
  for $u \in U$ **do**
    Update G_s by randomly selecting $\hat{d}_u$ distinct vertices from V and connecting u to each selected vertex
  // Edge adjustment: lower vertex degree constraint
  Calculate the degree difference Δd_V for V between the $\hat{d}_V$ and d_{Vs} (from G_s), and divide the lower vertices with $\Delta d_V > 0$ and $\Delta d_V < 0$ into two sets S_1 and S_2
  while $i < R$ *and* $flag = 1$ **do**
    $flag \leftarrow 0$
    for $u \in U$ **do**
      Calculate the number of edges d_{u1}, d_{u2} between u and S_1, S_2, respectively
      $s_u \leftarrow \min(|S_1| - d_{u1}, d_{u2})$
      if $s_u > 0$ **then**
        Update G_s by removing s_u edges between u and S_2, and adding s_u edges between u and distinct vertices in S_1 not currently adjacent to u
        Update $\Delta d_V, S_1, S_2, flag \leftarrow 1$
  return G_s;

4.2 Block-based Graph Synthesis (BKGS)

The DBGS method introduced in Section 4.1 effectively mitigates the impact of noise by preserving the degree information of both vertex sets. However, its utility may be limited when the perturbation noise is relatively low or the bipartite graph contains many high-degree vertices, as degree sequences alone provide limited information about the underlying connection patterns. Even when two graphs have similar degree distributions, their structural patterns can differ substantially. To address this, we further propose BKGS, a block-based bipartite graph synthesis method that captures additional information through graph partitioning. The overall procedure is presented in Algorithm 3.

Overview. In addition to perturbing the global degrees of the upper and lower vertices, BKGS partitions the bipartite graph into four blocks arranged in a 2×2 structure. However, a random partition may fail to reveal meaningful structural patterns, as the edges of the original graph may be distributed relatively evenly across the resulting blocks. To address this issue, we leverage the previously obtained global degree information to guide the partitioning process. Specifically, we use the perturbed global vertex degrees to divide the bipartite graph into regions that are expected to exhibit different levels of connectivity. We then extract degree information within each block to capture local structural patterns.

However, independently extracting degree information within each block requires additional privacy budget and may introduce inconsistencies between the block-level and global degree information. To address this, BKGS applies a consistency processing mechanism to align the degree information of individual blocks

with the previously obtained global degree information. Once the degree information across all blocks has been adjusted for consistency, BKGS reconstructs the bipartite graph within each block independently. The detailed procedure of BKGS is as follows.

Degree Perturbation. We first allocate a portion of the privacy budget (*i.e.*, $\varepsilon_1 = r_1 \cdot \varepsilon$) to obtain the perturbed global degree sequences of the bipartite graph using the DegPert function described in Algorithm 2. These perturbed degree sequences provide both a noisy characterization of the overall structure of the bipartite graph and essential information to guide subsequent partitioning.

Block Partitioning. This process begins by defining reference thresholds for the two vertex sets. Specifically, we use the average perturbed degrees of the upper and lower vertex sets as the respective thresholds, denoted by h_U and h_V. Based on these thresholds, we divide the upper and lower vertices into high-degree and low-degree groups, resulting in four blocks corresponding to the combinations of these groups.

In general, the block containing high-degree vertices on both sides is expected to be the densest, whereas the block containing low-degree vertices on both sides is expected to be the sparsest. Our goal is to further characterize the connectivity patterns within each block by examining its regional degree information. This is crucial to preserve the structural characteristics of the original graph.

As implemented in the BlockDiv function, we use the reference thresholds and perturbed vertex degrees to partition the bipartite graph into four blocks. We then apply the DegPert function independently to each block, allocating a privacy budget of $\varepsilon_2 = r_2 \cdot \varepsilon$ to each block to obtain its perturbed degree information.

Consistency Processing. Since the regional degree sequences obtained from individual blocks are perturbed independently, they may be inconsistent with the global perturbed degree sequences obtained in the first stage. To address this, BKGS adjusts the regional degree information to ensure consistency with the global perturbed degrees. Specifically, for each vertex u, we calculate each block's allocation weight by dividing the vertex's regional perturbed degree in that block by the sum of its regional perturbed degrees across the two corresponding blocks. We then multiply these proportions by the global perturbed degree of u and round the resulting values to obtain the final regional degree targets. This procedure ensures that the adjusted regional degrees align with the global perturbed degree of each vertex. We apply the same procedure to all vertices in both vertex sets.

Edge Generation. After obtaining the consistent regional degree information, we reconstruct the edges within each block using the DegRe function. Each block is reconstructed independently using its regional degree targets. Finally, we merge the reconstructed edges from all blocks to obtain the final synthetic bipartite graph.

Algorithm 3: Block-based graph synthesis (BKGS)

Input: Original graph $G = (U, V, E)$, privacy budget ε, the budget ratio for upper (lower) vertices r_u ($r_v = 1 - r_u$), the budget ratio for the global (regional) degree perturbation r_1 ($r_2 = 1 - r_1$), maximum number of rounds R.
Output: Synthetic bipartite graph G_s

Step 1: Degree Perturbation
$\hat{d}_U, \hat{d}_V \leftarrow \text{DegPert}(G, r_1 \cdot \varepsilon, r_u)$ // Global perturbation
Step 2: Block Partitioning
$h_U \leftarrow \text{Avg}(\hat{d}_U)$, $h_V \leftarrow \text{Avg}(\hat{d}_V)$
$S_1, S_2, S_3, S_4 \leftarrow \text{BlockDiv}(G, \hat{d}_U, \hat{d}_V, h_U, h_V)$
for $i \in \{1, 2, 3, 4\}$ **do**
  $\hat{d}^i_U, \hat{d}^i_V \leftarrow \text{DegPert}(S_i, r_2 \cdot \varepsilon, r_u)$ // Regional perturbation
Step 3: Consistency Processing
Redistribute all $\hat{d}^i_U$ and $\hat{d}^i_V$ so that, for each vertex, its regional degrees sum to its corresponding global perturbed degree in $\hat{d}_U$ or $\hat{d}_V$.
Step 4: Edge Generation
for $i \in \{1, 2, 3, 4\}$ **do**
  $U^i, V^i \leftarrow S_i$
  $G^i_s \leftarrow \text{DegRe}(U^i, V^i, \hat{d}^i_U, \hat{d}^i_V, R)$
Merge the edges generated for all blocks to obtain G_s

Function `BlockDiv`$(G, \hat{d}_U, \hat{d}_V, h_U, h_V)$**:**
  Initialize U_H, U_L, V_H, V_L as empty sets
  for $u \in U$ **do**
    if $\hat{d}_u < h_U$ **then**
      Add u into U_L
    else
      Add u into U_H
  for $v \in V$ **do**
    if $\hat{d}_v < h_V$ **then**
      Add v into V_L
    else
      Add v into V_H
  The edges between U_H and V_H form subgraph S_1, and the other three subgraphs S_2, S_3, and S_4 are formed in the same manner based on U_H, U_L, V_H, V_L
  return S_1, S_2, S_3, S_4;

4.3 Judgment-based Graph Synthesis (JBGS)

Although DBGS and BKGS perform well in certain scenarios, their fixed information extraction strategies make it difficult to perform consistently well across various settings. In practice, bipartite graphs exhibit diverse structural characteristics and may be subject to different levels of noise. Consequently, a fixed extraction strategy may not be suitable for all scenarios. To address this limitation, we propose JBGS, a judgment-based bipartite graph synthesis method that adaptively adjusts the granularity of information extraction according to the graph characteristics and noise level. Algorithm 4 presents the detailed procedure.

Overview. First, we allocate a small portion of the privacy budget ε_e to obtain a perturbed estimate of the total number of edges in the bipartite graph. Based on this estimate, we determine whether further partitioning is beneficial. Specifically, we proceed with partitioning only when the estimated information gain exceeds the estimated noise error for both vertex sets. If partitioning is selected, we further evaluate whether the block containing high-degree vertices on both sides warrants additional partitioning. This process

Algorithm 4: Judgment-based graph synthesis (JBGS)

Input: Original graph $G = (U, V, E)$, privacy budget ε, the budget ratio for upper (lower) vertices r_u ($r_v = 1 - r_u$), the budget ratio for the global (regional) degree perturbation r_1 ($r_2 = 1 - r_1$), maximum number of rounds R, gain coefficient λ.

Output: Synthetic bipartite graph G_s

Step 1: Edge Perturbation

$\varepsilon_e \leftarrow \min(0.001, 0.01\varepsilon), \varepsilon_r \leftarrow \varepsilon - \varepsilon_e$

$\tilde{m} \leftarrow |E| + Lap(\frac{1}{\varepsilon_e})$

Step 2: Partitioning Decision

Calculate the g_U^0, g_V^0 using Equation 4 and $\tilde{m}$

if $g_U^0 > 0$ *and* $g_V^0 > 0$ **then**

  Initialize $j \leftarrow 1, div \leftarrow 1, G_c \leftarrow G$

  while $div = 1$ **do**

    Calculate g_U^j, g_V^j using Equation 4, previously perturbed information ($\tilde{m}$ or $\hat{d}_{U_{j-1}}, \hat{d}_{V_{j-1}}$), and the current remaining privacy budget

    if $g_U^j > 0$ *and* $g_V^j > 0$ **then**

      $\hat{d}_{U_j}, \hat{d}_{V_j}, S_1^j, S_2^j, S_3^j, S_4^j \leftarrow$ InfoGet($G_c, \varepsilon_r, r_1, r_u, j$)

      for $i \in \{2, 3, 4\}$ **do**

        $\hat{d}^i_{U_j}, \hat{d}^i_{V_j} \leftarrow$ DegPert(S_i^j, $r_2 \cdot \varepsilon_r$, r_u)

      $G_c \leftarrow S_1^j, \varepsilon_r \leftarrow r_2 \cdot \varepsilon_r, j \leftarrow j + 1$

    else

      $\hat{d}_{U_j}, \hat{d}_{V_j}, S_1^j, S_2^j, S_3^j, S_4^j \leftarrow$ InfoGet($G_c, \varepsilon_r, r_1, r_u, j$)

      for $i \in \{1, 2, 3, 4\}$ **do**

        $\hat{d}^i_{U_j}, \hat{d}^i_{V_j} \leftarrow$ DegPert(S_i^j, $r_2 \cdot \varepsilon_r$, r_u)

      $div \leftarrow 0, J \leftarrow j$ // Stop block partitioning

else

  $\hat{d}_{U_0}, \hat{d}_{V_0} \leftarrow$ DegPert(G, ε_r, r_u)

Step 3: Consistency Processing

If partitioning was selected, adjust all $\hat{d}^i_{U_j}$ and $\hat{d}^i_{V_j}$ to ensure the edge sum of various blocks matches $\hat{d}_{U_0}$ and $\hat{d}_{V_0}$.

Step 4: Edge Generation

if $g_U^0 > 0$ *and* $g_V^0 > 0$ **then**

  $U_J^1, V_J^1 \leftarrow S_J^1$

  $G_{s_J}^1 \leftarrow$ DegRe($U_J^1, V_J^1, \hat{d}^1_{U_J}, \hat{d}^1_{V_J}, R$)

  for $j \in \{J, J-1, \cdots, 1\}$ **do**

    for $i \in \{2, 3, 4\}$ **do**

      $U_j^i, V_j^i \leftarrow S_i^j$

      $G_{s_j}^i \leftarrow$ DegRe($U_j^i, V_j^i, \hat{d}^i_{U_j}, \hat{d}^i_{V_j}, R$)

  Merge the edges generated for all blocks to obtain G_s

else

  $G_s \leftarrow$DegRe($U, V, \hat{d}_{U_0}, \hat{d}_{V_0}, R$)

Function `InfoGet`(G_c, ε_r, r_1, r_u, j)**:**

  $\hat{d}_{U_j}, \hat{d}_{V_j} \leftarrow$ DegPert($G_c, r_1 \cdot \varepsilon_r, r_u$)

  $h_{U_j} \leftarrow Avg(\hat{d}_{U_j}), h_{V_j} \leftarrow Avg(\hat{d}_{V_j})$

  $S_1^j, S_2^j, S_3^j, S_4^j \leftarrow$BlockDiv($G_c, \hat{d}_{U_j}, \hat{d}_{V_j}, h_{U_j}, h_{V_j}$)

  return $\hat{d}_{U_j}, \hat{d}_{V_j}, S_1^j, S_2^j, S_3^j, S_4^j$;

continues recursively until the expected information gain no longer justifies the additional noise introduced by finer-grained extraction. In this way, we obtain the perturbed global degree sequences and, when partitioning is selected, the regional degree information within the resulting blocks.

Since the regional degree information may be inconsistent with the perturbed global degree sequences, we proportionally redistribute the regional degrees to ensure consistency. Finally, we reconstruct the edges within each block and merge the results to obtain the final synthetic bipartite graph. The details of each step are presented below.

Edge Perturbation. Here, our goal is to use a small portion of the privacy budget to obtain global structural information that guides subsequent decisions. Since the total number of edges m reflects the overall density of the bipartite graph and has a sensitivity of 1, we perturb the edge count to obtain a noisy estimate from which the density can be calculated. This allows us to capture an important global characteristic while incurring only a small privacy cost.

Partitioning Decision. We dynamically determine the partitioning strategy using previously perturbed graph statistics and the available budget. Specifically, we evaluate whether further partitioning is beneficial by estimating the potential information gain and additional noise introduced by finer-grained degree extraction.

We next describe the information gain calculation for the upper vertices U; the same analysis applies to the lower vertices V. If we utilize the global average degree to approximate the degrees of all upper vertices, the total squared approximation error is given by

$$H_G = \sum_{u \in U} (d_u - \bar{d}_U)^2, \tag{1}$$

where d_u stands for the degree of vertex u in the original graph, and $\bar{d}_U$ stands for the average degree of the upper vertices. If the graph is partitioned into multiple blocks and the average degree within each block is used to approximate its regional vertex degrees, the total squared regional approximation error is

$$H_S = \sum_{g \in B} \sum_{u \in U_g} (d_{u_g} - \bar{d}_{U_g})^2, \tag{2}$$

where B denotes the set of blocks, d_{u_g} denotes the degree of vertex u within the block g, and $\bar{d}_{U_g}$ denotes the average degree of upper vertices in that block. Then, we can define the average information gain per upper vertex as $\Delta H_U = |H_G - H_S|/|U|$. This is because a larger ΔH_U indicates a greater discrepancy between the actual regional distribution and our expectation, suggesting that the partitioning reveals more informative structural differences.

However, it is difficult to directly calculate ΔH_U under DP constraints. Before making a partitioning decision, we can only access the noisy global information, such as the perturbed average degree. Observing that ΔH_U is usually proportional to the square of the degree, we approximate the information gain as follows:

$$\Delta H_U \approx \lambda \cdot (A_U)^2, \tag{3}$$

where $A_U = \text{Avg}(\hat{d}_U)$ is the estimated average degree of the upper vertices, and the parameter λ is a tunable coefficient that controls the estimated information gain.

We next estimate the noise introduced by degree perturbation. Since we use the Laplace mechanism to perturb vertex degrees, the

variance of the added noise is is $\frac{2\Delta f_d^2}{\varepsilon_{bu}^2}$, where ε_{bu} is the corresponding privacy budget allocated to perturbing the upper vertex degrees, and $\Delta f_d = 1$ is the sensitivity of the degree query. Thus, we use $N_U = \frac{2}{\varepsilon_{bu}^2}$ as the estimated noise impact per upper vertex. By comparing the estimated information gain with the estimated noise impact, we define the following judgment function:

$$g_U = \Delta H_U - N_U = \lambda \cdot (A_U)^2 - \frac{2}{\varepsilon_{bu}^2}. \tag{4}$$

Similarly, for the lower vertex set V, we obtain $g_V = \lambda \cdot (A_V)^2 - \frac{2}{\varepsilon_{bv}^2}$, where A_V is the estimated average degree of the lower vertices and ε_{bv} is the privacy budget allocated to perturbing their degrees.

If either g_U or g_V is negative, the estimated noise impact exceeds the potential information gain for at least one vertex set. In this case, further partitioning is considered unbeneficial, and we allocate the remaining budget to global degree perturbation without introducing additional blocks. Note that Line 9 in Algorithm 4 determines whether to further divide S_1^j into blocks, rather than G_c.

If both g_U and g_V are positive, the estimated information gain exceeds the corresponding noise impact on both sides. We therefore proceed with block partitioning to capture more detailed structural information.

When partitioning is selected, we first divide the bipartite graph into four blocks arranged in a 2×2 structure. We allocate a budget ratio of r_1 to perturb the global degree sequences and use the remaining ratio $r_2 = 1 - r_1$ for block-level degree extraction. Specifically, we apply the remaining budget to each of the three blocks other than the block containing high-degree vertices on both sides, while reserving the latter for potential further partitioning.

We then treat the high-degree block as a separate bipartite graph and assign it a remaining privacy budget of $r_2 \cdot \varepsilon_r$. Based on the perturbed information available for this block, we reevaluate whether further partitioning is beneficial. If the estimated information gain no longer exceeds the noise impact, we apply the same degree perturbation strategy to the sub-block consisting of high-degree nodes as we do to the other three sub-blocks. Otherwise, we recursively partition the block and repeat the same decision process. This procedure continues until the partitioning criterion indicates that further partitioning is no longer beneficial.

To evaluate g_U in Equation 4, we estimate A_U using the perturbed edge count $\tilde{m}$ or previously perturbed degree information, as described in the corresponding information extraction step in Line 11 of Algorithm 4. During recursive partitioning, the available privacy budget is updated at each stage. In particular, the budget for perturbing the upper vertex degrees is given by $\varepsilon_{bu} = r_1 \cdot r_u \cdot \varepsilon_r$. The budget for the lower vertices is determined in the same manner.

Through this adaptive procedure, we obtain the perturbed global degree sequences of the upper and lower vertices, together with the regional degree information of the resulting blocks whenever partitioning is selected. These outputs are then used in the subsequent consistency processing and edge generation stages.

Consistency Processing. If the graph is partitioned into multiple blocks in Step 2, the sum of the perturbed degrees of a vertex across its corresponding blocks may differ from its global perturbed degree. We therefore adopt the consistency processing mechanism

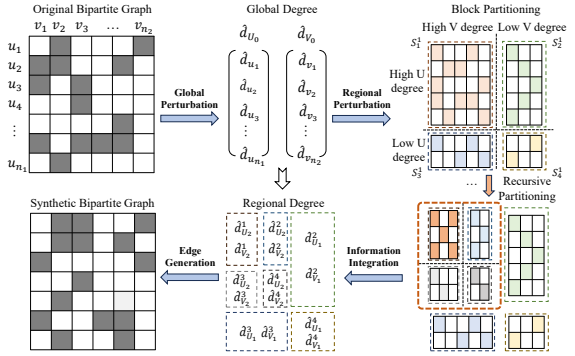


Figure 2: Pipeline of our proposed methods.

described in Section 4.2. Specifically, for each vertex in both vertex sets, we proportionally redistribute its regional perturbed degrees across its terminal blocks, with the degree in the last block set to the difference between its global perturbed degree and the sum of its degrees in the preceding blocks. This ensures that the regional degrees exactly match the global perturbed degree.

Edge Generation. If block partitioning is selected in Step 2, we first reconstruct the edges within each block according to its consistent regional degree information. We then merge the reconstructed edges from all blocks to obtain the final synthetic bipartite graph. Otherwise, we directly reconstruct the entire bipartite graph using the global perturbed degree sequences.

4.4 Summary

As shown in Figure 2, our proposed methods use up to four main steps: global perturbation, regional perturbation, information integration, and edge generation.

During the global perturbation phase, we perturb the degree sequences of the upper and lower vertex sets in the entire graph. In JBGS, we then determine whether further partitioning is beneficial by comparing the estimated information gain with the impact of noise. If the information gain outweighs the impact of noise, we proceed to regional perturbation. Specifically, we partition the bipartite graph into four blocks arranged in a 2×2 structure based on perturbed vertex degrees and perturb the degree information within each block. For the block containing high-degree vertices from both vertex sets, we further evaluate whether recursive partitioning is beneficial. This process continues until the estimated information gain no longer outweighs the estimated noise impact for at least one vertex set. Next, during information integration, we combine the global and regional degree information and adjust the regional degrees so that, for each vertex, their sum matches its global perturbed degree. Finally, we reconstruct the edges within each block based on the integrated degree information and merge the reconstructed edges to obtain the final synthetic bipartite graph.

Note that regional perturbation and information integration are not required in all scenarios. In JBGS, if the estimated noise impact is at least as large as the estimated information gain for either vertex set at the initial stage, we perturb only the global degree sequences and directly reconstruct the graph. A comparison of our proposed methods is provided in Table 1. DBGS perturbs only the global vertex degree sequences, without partitioning the graph. BKGS

Table 1: Comparison of three methods.

Method	Global Pert.	Regional Pert.	Information Inte.
DBGS	✓	×	×
BKGS	✓	Fixed partitioning	✓
JBGS	✓	Adaptive partitioning	✓

additionally partitions the graph into finer blocks using a fixed strategy that performs a single 2×2 partition. In contrast, JBGS adaptively determines whether and how to partition the graph based on the data characteristics and privacy budget. Since both BKGS and JBGS may incorporate degree information extracted at multiple granularities during graph partitioning, they require the information integration step to ensure consistency between global and regional degrees.

4.5 Algorithm Analysis

Privacy Analysis. We establish the privacy guarantees of the three proposed methods below.

Theorem 1. DBGS *satisfies ε-edge DP, where $\varepsilon = \varepsilon_U + \varepsilon_V$.*

Proof. In the degree perturbation step, we perturb the degree sequences of the upper and lower vertices using ε_U and ε_V, respectively. The degree restoration step does not access the true information. By sequential composition and the post-processing property of DP, DBGS satisfies ε-edge DP. □

Theorem 2. BKGS *satisfies ε-edge DP, where $\varepsilon = \varepsilon_1 + \varepsilon_2$.*

Proof. The privacy budget of BKGS is divided between global and regional degree perturbation, with $\varepsilon_1 + \varepsilon_2 = \varepsilon$.

In the global perturbation stage, BKGS perturbs the upper and lower global degree sequences using budgets $r_u \cdot \varepsilon_1$ and $r_v \cdot \varepsilon_1$, respectively, where $r_u + r_v = 1$. By sequential composition, the resulting global degree information satisfies ε_1-edge DP.

The block partition is determined solely from the perturbed global degree information. Hence, it is post-processing of the global perturbation output and incurs no additional privacy loss. In the regional perturbation stage, BKGS perturbs the degree information within each block using budgets $r_u \cdot \varepsilon_2$ for the upper vertices and $r_v \cdot \varepsilon_2$ for the lower vertices. Conditional on the previously released global degree information, the partition is fixed and the blocks contain disjoint subsets of possible edges. A change to one edge therefore affects the regional degree queries of at most one block. By sequential composition within each block and parallel composition across blocks, the joint regional release satisfies ε_2-edge DP.

Combining the global and regional perturbation stages through sequential composition yields an overall privacy guarantee of $(\varepsilon_1 + \varepsilon_2)$-edge DP. Finally, the consistency processing and edge generation stages use only the perturbed degree information and public parameters, so they incur no additional privacy loss. Thus, BKGS satisfies ε-edge DP. □

Theorem 3. JBGS *satisfies ε-edge DP.*

Proof. Initially, the privacy budget ε_e is used to perturb the edge count. The remaining budget $\varepsilon_r = \varepsilon - \varepsilon_e$ is used for subsequent stages. The partitioning decisions are made using perturbed edge count or previously released perturbed degree information. Therefore, the judgment function in Equation 4 and the resulting partitioning decisions are post-processing of previously released differentially private information and incur no additional privacy loss.

Next, we can analyze the budget allocation of ε_r in three various cases (*i.e.*, no block partitioning, a single partitioning operation, and multiple partitioning operations). The first case is similar to DBGS, and the second case is similar to BKGS. For the third case, under the recursive budget allocation and disjoint-block structure, the privacy loss from all degree perturbation stages is bounded by ε_r. Including the initial edge-count perturbation, the total privacy loss is at most $\varepsilon_e + \varepsilon_r = \varepsilon$.

Finally, the consistency processing and edge generation stages operate exclusively on the perturbed information. By the post-processing property of DP, they incur no additional privacy loss. Therefore, JBGS satisfies ε-edge DP. The detailed proof of Theorem 3 can be found in Appendix A.1 due to space constraints. □

Utility Analysis. In this part, we analyze the error introduced by the vertex degree perturbation.

Theorem 4 (Utility Bounds for Degree Perturbation). *Let $G = (U, V, E)$ be a bipartite graph, where $n_U = |U|$ and $n_V = |V|$. For each $u \in U$, let d_u denote its true degree. The perturbed degree is defined as $\hat{d}_u = \max\{0, \text{round}(d_u + Z_u)\}$, where $Z_u \sim Lap(0, 1/\varepsilon_U)$. Similarly, for each $v \in V$, $\hat{d}_v = \max\{0, \text{round}(d_v + Z_v)\}$, where $Z_v \sim Lap(0, 1/\varepsilon_V)$. Then the following bounds hold.*

(i) Expected error for an individual vertex. For every $u \in U$ and $v \in V$,

$$\mathbb{E}\left[|\hat{d}_u - d_u|\right] \le \frac{1}{2\sinh(\varepsilon_U/2)}, \tag{5}$$

$$\mathbb{E}\left[|\hat{d}_v - d_v|\right] \le \frac{1}{2\sinh(\varepsilon_V/2)}. \tag{6}$$

(ii) Expected ℓ_1 error of the degree sequences. Let

$$E_U = \sum_{u\in U} |\hat{d}_u - d_u|, \qquad E_V = \sum_{v\in V} |\hat{d}_v - d_v|. \tag{7}$$

Then

$$\mathbb{E}[E_U] \le \frac{n_U}{2\sinh(\varepsilon_U/2)}, \tag{8}$$

$$\mathbb{E}[E_V] \le \frac{n_V}{2\sinh(\varepsilon_V/2)}. \tag{9}$$

Consequently, the total degree error satisfies

$$\mathbb{E}[E_U + E_V] \le \frac{n_U}{2\sinh(\varepsilon_U/2)} + \frac{n_V}{2\sinh(\varepsilon_V/2)}. \tag{10}$$

(iii) High-probability maximum error bound. For any $\delta \in (0, 1)$, with probability at least $1 - \delta$, the following inequalities hold simultaneously:

$$\max_{u\in U} |\hat{d}_u - d_u| \le \frac{\ln(2n_U/\delta)}{\varepsilon_U} + \frac{1}{2}, \tag{11}$$

$$\max_{v\in V} |\hat{d}_v - d_v| \le \frac{\ln(2n_V/\delta)}{\varepsilon_V} + \frac{1}{2}. \tag{12}$$

The detailed proof of Theorem 4 is provided in Appendix A.2.

Computational Cost. Here, we analyze the computational cost of our proposed methods. Let $n_U = |U|$ and $n_V = |V|$ denote the number of upper and lower vertices, respectively, and let $N = n_U + n_V$. Let m denotes the number of edges in the synthetic graph. D

Table 2: Dataset statistics.

Dataset	$\|U\|$	$\|V\|$	$\|E\|$	Density	Type
Pantry [37]	3.2K	7.2K	68.2K	0.0029	User-Shopping
Yahoo [30]	4.4K	1.0K	0.1M	0.0279	User-Music
Movie [15]	6.0K	3.7K	1.0M	0.0447	User-Movie
Book [71]	8.0K	10.0K	0.9M	0.0112	User-Book

is the maximum recursive partitioning depth, and R is the maximum number of edge-correction rounds during degree restoration.

The computational costs of DBGS and BKGS are $O(N + Rm)$, while the computational cost of JBGS is $O\left(D(N + m) + Rm\right)$. The detailed introduction can be found in Appendix A.3.

5 EVALUATION

We introduce the experimental setup in Section 5.1. In Section 5.2, we first conduct an end-to-end experiment to illustrate the effectiveness of our methods compared to existing methods for general graph synthesis. Next, we perform ablation experiments to explore the influence of various components in Section 5.3, and examine the effects of parameter settings in Section 5.4. Then, we evaluate downstream utility through a practical application in Section 5.5. We further explore the scalability of different methods in Section 5.6.

5.1 Experimental Setup

5.1.1 Datasets. We conduct the experiments on four real-world datasets. The detailed statistics are provided in Table 2. $|U|$ and $|V|$ denote the numbers of upper and lower vertices, and $|E|$ denotes the number of edges. Pantry [37] is a review dataset for Amazon's prime pantry. Yahoo [30] contains music-listening information collected by Yahoo. Movie [15] is a classic movie-rating dataset. Book [71] is a comment dataset for book recommendation.

5.1.2 Metrics. To comprehensively evaluate the utility of synthetic bipartite graphs, we use metrics covering four aspects: node distribution, spectral feature, global connectivity, and local structure. These metrics are also commonly used in existing studies [7, 18, 19, 36, 70].

For node distribution, we calculate the total variation distance between the original and synthetic degree distributions separately for each vertex set and define degree similarity as one minus this distance. For spectral feature, we construct a symmetric adjacency matrix for the bipartite graph, and use the k (*i.e.*, 20) eigenvalues with the largest absolute values, to form a spectral signature. We then calculate spectral similarity. For the above metrics, a value closer to 1 indicates a higher similarity.

For global connectivity, we calculate the relative error (RE) of graph density. For local structure, we measure the REs of butterfly count and average common neighbor count. For these three metrics, a value closer to 0 indicates greater similarity.

5.1.3 Baselines. To our knowledge, there are no other differentially private synthesis approaches tailored for bipartite graphs. Therefore, we adopt several differentially private synthesis methods for the general graphs [7, 36, 69], along with Naive introduced in Section 3, as baselines. TmF [36] perturbs each entry of the original adjacency matrix by adding Laplace noise and selects the top-m entries in the perturbed matrix as edges. DER [7] explores dense regions in the adjacency matrix and reconstructs the edges using the exponential mechanism. As the state-of-the-art method, PrivGraph [69] divides all vertices into various communities, extracts the information both within and across these communities, and integrates this information to reconstruct the entire graph. By considering the bipartite graph as a general graph, these baselines for general graphs can be directly applied to this problem.

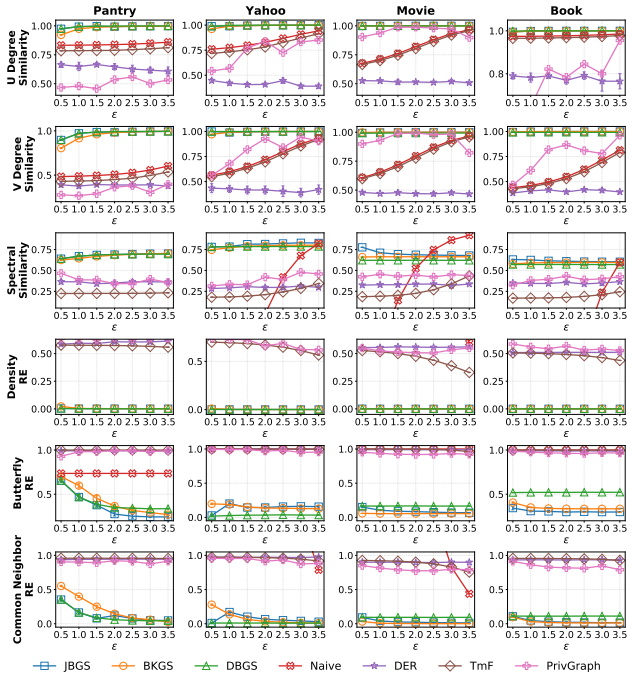


Figure 3: End-to-end comparison. The columns represent the used datasets, and the rows stand for different metrics. For the first three rows, higher is better. For the last three rows, lower is better.

5.1.4 Experimental Settings. We explore the performance of various methods with a total privacy budget ε ranging from 0.5 to 3.5. The gain coefficient λ is set to 0.1, and the maximum number of rounds R is set to 50. For the budget ratios, we set $r_1 = r_u = 0.5$. For the baselines, we adopt the default parameters from the original papers. The experiments are run on a server with an Intel Xeon Gold 6330 processor and 1TB of memory. We repeat each experiment 10 times for each setting, and report the mean and standard variance.

5.2 End-to-End Evaluation

In this section, we perform an end-to-end evaluation of various methods on six metrics. Figure 3 illustrates the experimental results on four different datasets. We obtain the following observations.

- As the privacy budget ε increases, the quality of synthetic graphs gradually improves, becoming closer to the original graphs. Our proposed methods (*i.e.*, DBGS, BKGS and JBGS) significantly outperform the baselines across most metrics and datasets.
- DBGS can achieve high-quality synthesis under severe noise perturbation or on sparse graphs, since vertex degree can effectively balance noise impact and information loss. BKGS tends to perform better on denser graphs or under higher privacy budgets since block partitioning can capture more information in

these cases. JBGS adaptively adjusts the partitioning strategy according to the graph characteristics and the privacy budget, which can flexibly meet the requirements of various scenarios.

- Naive tends to introduce excessive edges, resulting in particularly poor performance across multiple metrics under small privacy budgets. As ε increases, its performance improves on certain metrics, particularly for denser graphs (*e.g.*, Movie), where the effect of edge inflation is less pronounced.
- The baseline methods designed for general graph synthesis perform poorly on the evaluated bipartite graphs, since they do not adequately account for bipartite structure.

5.3 Ablation Study

In this section, we perform ablation experiments to assess the effects of different components in JBGS. Recalling the DegRe function, edge connections are adjusted to satisfy degree constraints of both vertex sets. To evaluate the effect of jointly constraining both vertex sets, we consider a variant termed "without Bi-level Constraints," which retains the original edge initialization based on upper vertex degrees but adjusts edges to match lower vertex degree targets without preserving the upper vertex degrees. Moreover, as described in Algorithm 4, the consistency processing step reconciles the regional degree information across blocks with the global perturbed degrees. We therefore investigate its impact by considering another variant, "without Consistency Processing," which omits this step.

Figure 4 presents the ablation results across four evaluation metrics. Compared with "without Bi-level Constraints," JBGS achieves better performance on three metrics, with lower vertex degree similarity being the exception. Since the variant considers only lower vertex degree constraints during edge adjustment, it achieves higher similarity in the lower vertex degree distribution. However, relaxing the upper vertex degree constraints compromises its performance on the other metrics. These results demonstrate the benefits of jointly considering degree constraints on both vertex sets.

The results for the "without Consistency Processing" variant show that omitting consistency processing leads to poorer performance in terms of degree similarity and density RE. This degradation is due to inconsistencies between regional degree estimates and the global perturbed degrees, which can accumulate across blocks and distort the overall degree distribution and graph density. Interestingly, the variant achieves a lower butterfly-count RE than the complete JBGS method. One possible explanation is that consistency processing may alter local structures within individual blocks when reconciling regional and global degree information, thereby affecting butterfly counts. These findings suggest that consistency processing improves certain properties but does not necessarily benefit all evaluation metrics.

5.4 Parameter Variation

In this part, we explore the impact of various parameter settings (*i.e.*, the gain coefficient, the maximum number of degree-restoration rounds, and privacy budget allocation) on JBGS to inform practical deployment.

5.4.1 Impact of Gain Coefficient. Recalling Equation 3, we utilize a gain coefficient λ to estimate the information gain. A suitable λ is crucial for enhancing the utility since it directly influences the

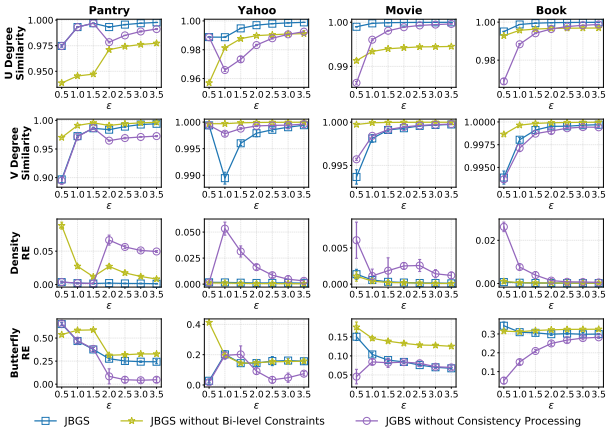


Figure 4: Impact of bi-level constraints and consistency processing. For the first two rows, higher is better. For the last two rows, lower is better.

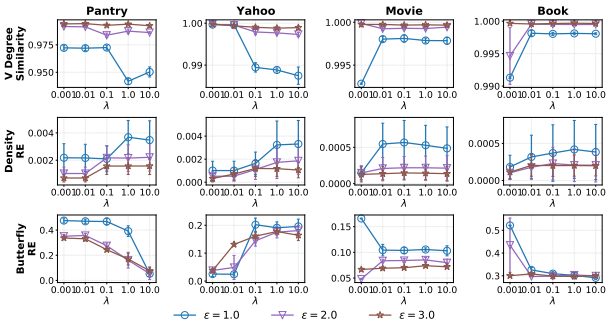


Figure 5: Impact of various gain coefficients λ. For the first row, higher is better. For the last two rows, lower is better.

partitioning decisions. Figure 5 illustrates the impact of various λ on three evaluation metrics. We make the following observations.

First, λ has an obvious impact on these metrics, particularly under small privacy budgets (*e.g.*, $\varepsilon = 1.0$). When λ is too small, the information gain tends to be underestimated, making JBGS less likely to partition the graph. Consequently, this setting may fail to capture useful regional structural information, leading to degraded performance in many cases. Conversely, an excessively large λ can overestimate the potential information gain, encouraging more frequent partitioning and potentially exposing the synthesis process to greater noise interference.

Moreover, the effects of λ vary across datasets and evaluation metrics. For example, its impact on degree similarity and butterfly-count error exhibits opposite trends on the Yahoo and Book datasets. Even for the same dataset and metric, the performance trends associated with different λ values are not fully consistent across privacy budgets. These observations suggest that no single value of λ is uniformly optimal across all datasets, metrics, and privacy budgets. We find that JBGS maintains competitive synthesis quality when $\lambda = 0.1$, thus we set $\lambda = 0.1$ throughout the experiments.

5.4.2 Impact of Maximum Round. Recalling DegRe function used for degree restoration, we impose a maximum traversal round R for the edge adjustment. Intuitively, a small R may leave larger discrepancies from the lower vertex degree targets, reducing the

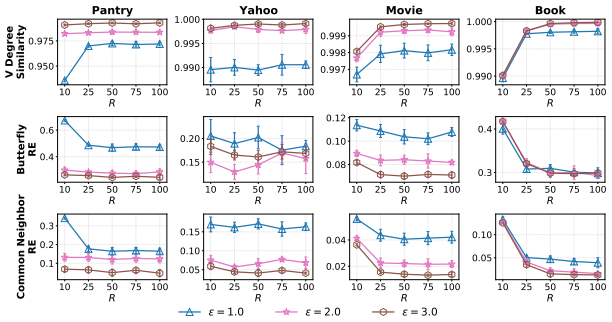


Figure 6: Impact of various maximum rounds R. For the first row, higher is better. For the last two rows, lower is better.

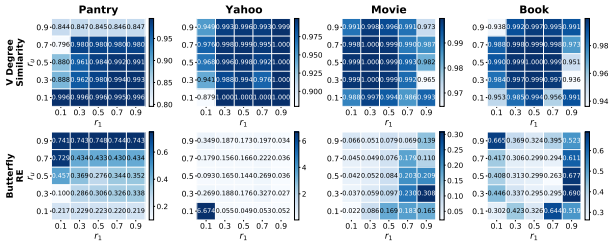


Figure 7: Impact of budget allocation when $\varepsilon = 2.0$. For the first row, higher is better. For the last row, lower is better.

quality of the synthetic graph. In contrast, an excessively large R may incur unnecessary computational overhead.

Figure 6 illustrates the impact of various values of R on three metrics. Overall, as R increases, most metrics show a trend closer to the real data, demonstrating the important role of bi-level degree constraints in improving the utility of the bipartite graph. We also observe that R has a pronounced impact under smaller privacy budgets than under larger ones. For instance, when the privacy budget ε is 2.0 or 3.0, the performance differences among different values of R on the Pantry dataset are relatively small. In contrast, when $\varepsilon = 1.0$, the choice of R leads to substantially different results.

At the same time, we also find that on some datasets with denser connections (*i.e.*, Yahoo and Movie), increasing R does not consistently improve all metrics and may even degrade certain aspects of utility. This might be because many edge configurations are compatible with the same degree constraints in these denser graphs. Consequently, additional adjustment rounds may lead to a graph that more closely matches the degree targets but is less faithful to other structural properties of the original graph.

Overall, we find that $R = 50$ yields promising results across the evaluated metrics and datasets, while further increasing R provides limited additional utility. Therefore, we set $R = 50$ throughout our experiments to balance synthesis quality and computational cost.

5.4.3 Impact of Budget Allocation. Recalling Section 4.3, we allocate an r_1 fraction of the remaining privacy budget ε_r to degree perturbation and the remaining $r_2 = 1 - r_1$ fraction to regional degree extraction or further recursive processing. During the degree perturbation process, we allocate an r_u fraction of the corresponding budget to the upper vertices and the remaining $r_v = 1 - r_u$ fraction to the lower vertices. Here, we vary r_1 and r_u from 0.1 to 0.9 in increments of 0.2. Figure 7 illustrates the impact of budget allocation on two metrics across four datasets when ε is 2.0. We make the following observations.

First, increasing r_1 yields different effects on various metrics. As r_1 increases, the degree similarity of lower vertices gradually decreases while the butterfly-count RE initially decreases and then increases. This behavior reflects the trade-off between accurate current-block degree estimation and regional degree extraction. A larger r_1 allocates more budget to degree perturbation, potentially improving the accuracy of those perturbed degree sequences. However, it leaves a smaller budget fraction r_2 for regional degree extraction or further recursive processing, which may reduce the accuracy of the regional structural information and increase butterfly-count RE.

Second, in most cases, setting r_u too high or too low degrades the utility. This is because the degree information of both upper and lower vertices contributes to the quality of the synthetic graph. An imbalanced allocation may lead to less accurate degree estimates for one vertex set, thereby compromising the overall fidelity of the generated graph. Based on the above analysis, we choose to use $r_1 = r_u = 0.5$ to achieve a great balance in various aspects.

5.5 Case Study: Link Prediction

In this section, we explore the performance of various methods on the link prediction task, which is widely used in our daily life (*e.g.*, recommender system). This task aims to infer the missing or unknown edges from available information [78].

Specifically, we hold out a portion (*i.e.*, 20%) of the edges from each original bipartite graph as positive test samples and construct negative samples from upper–lower vertex pairs that are not connected in the original bipartite graph. We then apply the same link prediction method [79] to the remaining parts of the original graph and each synthetic graph, and evaluate their prediction performance on the same test samples using the area under the receiver operating characteristic curve (AUC). A higher AUC indicates a stronger ability to distinguish existing edges from non-existent ones.

Figure 8 illustrates the AUC scores of different methods on the Pantry and Yahoo datasets. On the relatively sparse Pantry dataset, the performance of our methods closely approaches that obtained using the original graph, represented by the black dashed line, and even slightly surpasses it under larger privacy budgets. In contrast, most baseline methods achieve AUC scores close to 0.5, which corresponds to random guessing. This suggests that these baseline methods struggle to preserve sufficient structural information for link prediction on sparse bipartite graphs. On the Yahoo dataset, our methods achieve AUC scores below those obtained using the original graph but substantially outperform the baselines. We also observe that most baseline methods improve as the privacy budget increases. This can be attributed to the higher density of the Yahoo dataset, which results in a higher proportion of ground-truth edges among the perturbed edges.

Overall, these results demonstrate that our proposed methods preserve useful structural information for downstream link prediction and achieve robust performance on the two evaluated datasets, even under strong privacy constraints.

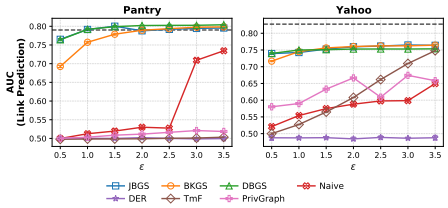


Figure 8: AUC comparison for link prediction.

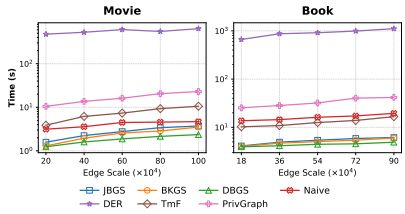


Figure 9: Scalability evaluation.

5.6 Scalability

We further evaluate the scalability of all methods by varying the edge scale of the bipartite graph datasets. The running times on the Movie and Book datasets are reported in Figure 9.

The results show that our three proposed methods consistently outperform the baselines in terms of running time across all evaluated edge scales. Among our methods, DBGS achieves the shortest running time because it only requires global degree perturbation, followed by BKGS. JBGS incurs the highest computational cost among the three, as it may perform multiple levels of recursive block partitioning. Among the baseline methods, those involving complex data encoding generally require longer processing times. For instance, DER has a substantially longer running time than the other methods since it constructs a tree structure and performs extensive traversal and exploration.

Overall, our methods synthesize the evaluated bipartite graphs in less than 10 seconds, with running times at least an order of magnitude shorter than those of some baseline methods. The performance of our methods is also stable, and the processing time does not increase sharply with the growth of edge scales. These results demonstrate the efficiency and scalability of our methods over the evaluated graph sizes.

6 RELATED WORK

6.1 Graph Privacy Protection

There are a series of studies focus on the specific graph task under DP [27, 34, 40, 41, 43, 45, 50, 60, 62, 72, 74]. In particular, Hay *et al.* [16] investigate the estimation of degree distributions in private networks. Day *et al.* [9] further explore the degree distribution publishing under node DP. Karwa *et al.* [23] design two algorithms to release answers to subgraph counting queries under edge DP. Using the Laplace and exponential mechanisms, Wang *et al.* [58] further propose two methods to compute the eigendecomposition of the adjacency matrix while satisfying differential privacy. Mulle *et al.* [35] design a graph clustering approach under edge DP.

In addition, there are also some works focus on the graph privacy protection under local differential privacy (LDP) settings [20, 26, 28, 39, 59, 64, 66, 80]. In [21], Imola *et al.* present the algorithms for counting subgraphs with LDP. He *et al.* [18] introduce a multi-round interactive algorithm for counting butterflies in bipartite graphs while satisfying edge LDP. They also study the problem of estimating the number of common neighbors of two vertices in the same vertex set under edge LDP [19]. Sun *et al.* [47] design a multi-phase private framework that enables an analyst to accurately estimate subgraph counts. In [65], Ye *et al.* present an LDP-enabled graph metric estimation framework.

6.2 DP Data Synthesis

Beyond graph data synthesis, there is also a substantial body of research on synthesizing other types of data under DP. Recently, differentially private tabular data synthesis has garnered extensive research attention. In general, there are three mainstream solutions for protecting tabular data: graphical model-based, game-based, and deep generative model-based. The graphical model-based methods aim to estimate a graphical model which approximates the distribution of the original dataset under DP [1, 3, 4, 31, 32, 73, 77]. The game-based schemes formulate data synthesis as a zero-sum game [13, 51]. The deep generative model-based approaches first train a deep generative model that satisfies DP and then adopt the deep generative model to generate a synthetic dataset [2, 5, 12, 75].

Moreover, there are some studies that explore the synthesis of trajectory dataset under DP [33, 46, 48, 49, 55]. Chen *et al.* [6] propose to build a prefix tree to describe the n-gram model and design an adaptive privacy allocation strategy. He *et al.* [17] further propose DPT, which discretizes space at multiple granularities and builds multiple prefix trees. In [14], AdaTrace is introduced to enforce deterministic constraints on the generated trajectories while satisfying DP, thereby enhancing the algorithm's resilience against common threats. In [53], the PrivTrace method involving dynamic selection of first-order and second-order models is introduced to balance noise and correlation errors.

7 CONCLUSION

In this paper, we present the first systematic study of differentially private bipartite graph synthesis. First, we propose a degree-based bipartite graph synthesis method, DBGS, which mitigates the noise effects using degree information from both vertex sets and employs a sequential optimization strategy to improve reconstruction accuracy. To better exploit structural information under lower noise levels, we further develop a block-based synthesis method, BKGS, which partitions the bipartite graph into multiple blocks and leverages regional degree information to enhance graph reconstruction. To accommodate diverse data characteristics and privacy requirements, we further propose a judgment-based synthesis method, JBGS, which adaptively adjusts its partitioning strategy according to the graph characteristics and privacy budget. Extensive experiments on four real-world bipartite graph datasets demonstrate the superiority of our proposed methods. In addition, we conduct comprehensive ablation studies and parameter sensitivity analyses to investigate the contributions of different components and the effects of key parameter choices. We further show the advantage of our methods on a specific case study.

REFERENCES

[1] Ergute Bao, Xiaokui Xiao, Jun Zhao, Dongping Zhang, and Bolin Ding. 2021. Synthetic Data Generation with Differential Privacy via Bayesian Networks. *Journal of Privacy and Confidentiality* (2021).

[2] Brett K Beaulieu-Jones, Zhiwei Steven Wu, Chris Williams, Ran Lee, Sanjeev P Bhavnani, James Brian Byrd, and Casey S Greene. 2019. Privacy-Preserving Generative Deep Neural Networks Support Clinical Data Sharing. *Circulation: Cardiovascular Quality and Outcomes* 12, 7 (2019), e005122.

[3] Vincent Bindschaedler, Reza Shokri, and Carl A Gunter. 2017. Plausible Deniability for Privacy-Preserving Data Synthesis. *PVLDB* 10, 5 (2017), 481–492.

[4] Kuntai Cai, Xiaoyu Lei, Jianxin Wei, and Xiaokui Xiao. 2021. Data Synthesis via Differentially Private Markov Random Fields. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2190–2202.

[5] Kai Chen, Xiaochen Li, Chen Gong, Ryan Mckenna, and Tianhao Wang. 2025. Benchmarking Differentially Private Tabular Data Synthesis: [Experiments & Analysis]. *Proceedings of the ACM on Management of Data* (2025).

[6] Rui Chen, Gergely Acs, and Claude Castelluccia. 2012. Differentially Private Sequential Data Publication via Variable-Length N-Grams. In *CCS*. 638–649.

[7] Rui Chen, Benjamin Fung, Philip S Yu, and Bipin C Desai. 2014. Correlated Network Data Publication via Differential Privacy. *The VLDB Journal* 23, 4 (2014), 653–676.

[8] Xian Chen, Ruobing Bai, and Pan Peng. 2026. Differentially Private Range Subgraph Counting. In *ICML*.

[9] Wei-Yen Day, Ninghui Li, and Min Lyu. 2016. Publishing Graph Degree Distribution with Node Differential Privacy. In *Proceedings of the 2016 International Conference on Management of Data*. 123–138.

[10] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *Theory of Cryptography Conference*. Springer, 265–284.

[11] Cynthia Dwork, Aaron Roth, et al. 2014. The Algorithmic Foundations of Differential Privacy. *Foundations and Trends® in Theoretical Computer Science* 9, 3–4 (2014), 211–407.

[12] Lorenzo Frigerio, Anderson Santana de Oliveira, Laurent Gomez, and Patrick Duverger. 2019. Differentially Private Generative Adversarial Networks for Time Series, Continuous, and Discrete Open Data. In *IFIP International Conference on ICT Systems Security and Privacy Protection*.

[13] Marco Gaboardi, Emilio Jesús Gallego Arias, Justin Hsu, Aaron Roth, and Zhiwei Steven Wu. 2014. Dual Query: Practical Private Query Release for High Dimensional Data. In *ICML*. PMLR, 1170–1178.

[14] Mehmet Emre Gursoy, Ling Liu, Stacey Truex, Lei Yu, and Wenqi Wei. 2018. Utility-Aware Synthesis of Differentially Private and Attack-Resilient Location Traces. In *CCS*. 196–211.

[15] F Maxwell Harper and Joseph A Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4 (2015), 1–19.

[16] Michael Hay, Chao Li, Gerome Miklau, and David Jensen. 2009. Accurate Estimation of the Degree Distribution of Private Networks. In *ICDM*.

[17] Xi He, Graham Cormode, Ashwin Machanavajjhala, Cecilia M Procopiuc, and Divesh Srivastava. 2015. DPT: Differentially Private Trajectory Synthesis Using Hierarchical Reference Systems. *PVLDB* 8, 11 (2015), 1154–1165.

[18] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, Wei Ni, and Ying Zhang. 2024. Butterfly Counting Over Bipartite Graphs with Local Differential Privacy. In *ICDE*. 2351–2364.

[19] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, and Ying Zhang. 2024. Common Neighborhood Estimation Over Bipartite Graphs under Local Differential Privacy. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–26.

[20] Yihua Hu, Kuncan Wang, and Wei Dong. 2026. Acyclic Graph Pattern Counting under Local Differential Privacy. *SIGMOD* (2026).

[21] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2021. Locally Differentially Private Analysis of Graph Statistics. In *30th USENIX Security Symposium (USENIX Security 21)*. 983–1000.

[22] Tianxi Ji, Pan Li, Emre Yilmaz, Erman Ayday, Yanfang Ye, and Jinyuan Sun. 2021. Differentially Private Binary-and Matrix-Valued Data Query: An XOR Mechanism. *Proceedings of the VLDB Endowment* 14, 5 (2021), 849–862.

[23] Vishesh Karwa, Sofya Raskhodnikova, Adam Smith, and Grigory Yaroslavtsev. 2011. Private Analysis of Graph Structure. *Proceedings of the VLDB Endowment* 4, 11 (2011), 1146–1157.

[24] Yang Li, Michael Purcell, Thierry Rakotoarivelo, David Smith, Thilina Ranbaduge, and Kee Siong Ng. 2023. Private Graph Data Release: A Survey. *Comput. Surveys* 55, 11 (2023), 1–39.

[25] Zhao Li, Xin Shen, Yuhang Jiao, Xuming Pan, Pengcheng Zou, Xianling Meng, Chengwei Yao, and Jiajun Bu. 2020. Hierarchical Bipartite Graph Neural Networks: Towards Large-scale E-commerce Applications. In *ICDE*.

[26] Wanyu Lin, Baochun Li, and Cong Wang. 2022. Towards Private Learning on Decentralized Graphs With Local Differential Privacy. *IEEE Transactions on Information Forensics and Security* (2022), 2936–2946.

[27] Shang Liu, Yang Cao, Takao Murakami, Weiran Liu, Seng Pei Liew, Tsubasa Takahashi, Jinfei Liu, and Masatoshi Yoshikawa. 2026. Federated Graph Analytics with Differential Privacy. *IEEE Transactions on Dependable and Secure Computing* (2026).

[28] Shang Liu, Yang Cao, Takao Murakami, and Masatoshi Yoshikawa. 2022. A Crypto-Assisted Approach for Publishing Graph Statistics with Node Local Differential Privacy. In *Big Data*.

[29] Shang Liu, Hao Du, Yang Cao, Bo Yan, Jinfei Liu, and Masatoshi Yoshikawa. 2025. PGB: Benchmarking Differentially Private Synthetic Graph Generation Algorithms. In *ICDE*. 1348–1361.

[30] Benjamin M Marlin and Richard S Zemel. 2009. Collaborative Prediction and Ranking with Non-Random Missing Data. In *ACM RecSys*.

[31] Ryan McKenna, Brett Mullins, Daniel Sheldon, and Gerome Miklau. 2022. Aim: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data. *PVLDB* 15, 11 (2022), 2599–2612.

[32] Ryan McKenna, Daniel Sheldon, and Gerome Miklau. 2019. Graphical-Model Based Estimation and Inference for Differential Privacy. In *International Conference on Machine Learning*. PMLR, 4435–4444.

[33] Àlex Miranda-Pascual, Patricia Guerra-Balboa, Javier Parra-Arnau, Jordi Forné, and Thorsten Strufe. 2023. SoK: Differentially Private Publication of Trajectory Data. *Proceedings on Privacy Enhancing Technologies* (2023).

[34] Tamara T Mueller, Johannes C Paetzold, Chinmay Prabhakar, Dmitrii Usynin, Daniel Rueckert, and Georgios Kaissis. 2022. Differentially Private Graph Neural Networks for Whole-Graph Classification. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 6 (2022), 7308–7318.

[35] Yvonne Mülle, Chris Clifton, and Klemens Böhm. 2015. Privacy-Integrated Graph Clustering Through Differential Privacy. In *EDBT/ICDT Workshops*, Vol. 157.

[36] Hiep H Nguyen, Abdessamad Imine, and Michaël Rusinowitch. 2015. Differentially Private Publication of Social Graphs at Linear Cost. In *ASONAM*.

[37] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects. In *EMNLP-IJCNLP*. 188–197.

[38] Jianwei Qian, Xiang-Yang Li, Chunhong Zhang, Linlin Chen, Taeho Jung, and Junze Han. 2017. Social Network De-Anonymization and Privacy Inference with Knowledge Graph Model. *IEEE Transactions on Dependable and Secure Computing* 16, 4 (2017), 679–692.

[39] Zhan Qin, Ting Yu, Yin Yang, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2017. Generating Synthetic Decentralized Social Graphs with Local Differential Privacy. In *CCS*. 425–438.

[40] Xun Ran, Qingqing Ye, Haibo Hu, Xin Huang, Jianliang Xu, and Jie Fu. 2024. Differentially Private Graph Neural Networks for Link Prediction. In *ICDE*. 1632–1644.

[41] Sofya Raskhodnikova and Adam Smith. 2016. Lipschitz Extensions for Node-Private Graph Statistics and the Generalized Exponential Mechanism. In *FOCS*.

[42] Ryan A Rossi, Brian Gallagher, Jennifer Neville, and Keith Henderson. 2013. Modeling Dynamic Behavior in Large Evolving Graphs. In *WSDM*. 667–676.

[43] Sina Sajadmanesh, Ali Shahin Shamsabadi, Aurélien Bellet, and Daniel Gatica-Perez. 2023. GAP: Differentially Private Graph Neural Networks with Aggregation Perturbation. In *32nd USENIX Security Symposium*. 3223–3240.

[44] Alessandra Sala, Xiaohan Zhao, Christo Wilson, Haitao Zheng, and Ben Y Zhao. 2011. Sharing Graphs Using Differentially Private Graph Models. In *SIGCOMM*. 81–98.

[45] Entong Shen and Ting Yu. 2013. Mining Frequent Graph Patterns with Differential Privacy. In *KDD*. 545–553.

[46] Danyu Sun, Joann Qiongna Chen, Chen Gong, Tianhao Wang, and Zhou Li. 2024. NetDPSyn: Synthesizing Network Traces under Differential Privacy. In *Proceedings of the 2024 ACM on Internet Measurement Conference*. 545–554.

[47] Haipei Sun, Xiaokui Xiao, Issa Khalil, Yin Yang, Zhan Qin, Hui Wang, and Ting Yu. 2019. Analyzing Subgraph Statistics from Extended Local Views with Decentralized Differential Privacy. In *CCS*.

[48] Xinyue Sun, Qingqing Ye, Haibo Hu, Yuandong Wang, Kai Huang, Tianyu Wo, and Jie Xu. 2023. Synthesizing Realistic Trajectory Data with Differential Privacy. *IEEE Transactions on Intelligent Transportation Systems* (2023).

[49] Shun Takagi, Li Xiong, Fumiyuki Kato, Yang Cao, and Masatoshi Yoshikawa. 2024. HRNet: Differentially Private Hierarchical and Multi-Resolution Network for Human Mobility Data Synthesization. *Proc. VLDB Endow.* (2024), 3058–3071.

[50] Jalaj Upadhyay, Sarvagya Upadhyay, and Raman Arora. 2021. Differentially Private Analysis on Graph Streams. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 1171–1179.

[51] Giuseppe Vietri, Grace Tian, Mark Bun, Thomas Steinke, and Steven Wu. 2020. New Oracle-Efficient Algorithms for Private Synthetic Data Release. In *ICML*. PMLR, 9765–9774.

[52] Hanchen Wang, Kai Wang, Wenjie Zhang, and Ying Zhang. 2024. Bipartite Graph Analytics: Current Techniques and Future Trends. In *ICDE*.

[53] Haiming Wang, Zhikun Zhang, Tianhao Wang, Shibo He, Michael Backes, Jiming Chen, and Yang Zhang. 2023. PrivTrace: Differentially Private Trajectory Synthesis by Adaptive Markov Model. In *USENIX Security Symposium*.

[54] Mengyuan Wang, Hongbo Jiang, Peng Peng, Youhuan Li, and Wenbin Huang. 2024. Toward Accurate Butterfly Counting with Edge Privacy Preserving in Bipartite Networks. In *IEEE INFOCOM*. 2289–2298.
[55] Nana Wang and Mohan Kankanhalli. 2026. DPTraj-PM: Differentially Private Trajectory Synthesis Using Prefix Tree and Markov Process. *ACM Trans. Spatial Algorithms Syst.* (2026).
[56] Yansheng Wang, Yongxin Tong, Cheng Long, Pan Xu, Ke Xu, and Weifeng Lv. 2019. Adaptive Dynamic Bipartite Graph Matching: A Reinforcement Learning Approach. In *ICDE*.
[57] Yue Wang, Xintao Wu, and Donghui Hu. 2016. Using Randomized Response for Differential Privacy Preserving Data Collection. In *EDBT/ICDT Workshops*, Vol. 1558. 0090–6778.
[58] Yue Wang, Xintao Wu, and Leting Wu. 2013. Differential Privacy Preserving Spectral Graph Analysis. In *PAKDD*. 329–340.
[59] Chengkun Wei, Shouling Ji, Changchang Liu, Wenzhi Chen, and Ting Wang. 2020. AsgLDP: Collecting and Generating Decentralized Attributed Graphs with Local Differential Privacy. *IEEE Transactions on Information Forensics and Security* 15 (2020), 3239–3254.
[60] Jianxin Wei, Yizheng Zhu, Xiaokui Xiao, Ergute Bao, Yin Yang, Kuntai Cai, and Beng Chin Ooi. 2025. GCON: Differentially Private Graph Convolutional Network via Objective Perturbation. In *ICDE*.
[61] Ziyao Wei, Qing Liu, Zhikun Zhang, Shouling Ji, and Yunjun Gao. 2025. Privacy-Preserving Triangle Counting in Directed Graphs. In *ICDE*. 2588–2600.
[62] Siyuan Xia, Beizhen Chang, Karl Knopf, Yihan He, Yuchao Tao, and Xi He. 2021. DPGraph: A Benchmark Platform for Differentially Private Graph Analysis. In *Proceedings of the 2021 International Conference on Management of Data*. 2808–2812.
[63] Qian Xiao, Rui Chen, and Kian-Lee Tan. 2014. Differentially Private Network Data Release via Structural Inference. In *KDD*. 911–920.
[64] Wen Xu, Pengpeng Qiao, Shang Liu, Zhirun Zheng, Yang Cao, and Zhetao Li. 2025. Continuous Publication of Weighted Graphs with Local Differential Privacy. *PVLDB* 18, 11 (2025), 4214–4226.
[65] Qingqing Ye, Haibo Hu, Man Ho Au, Xiaofeng Meng, and Xiaokui Xiao. 2020. LF-GDPR: A Framework for Estimating Graph Metrics with Local Differential Privacy. *TKDE* (2020).
[66] Qingqing Ye, Haibo Hu, Man Ho Au, Xiaofeng Meng, and Xiaokui Xiao. 2020. Towards Locally Differentially Private Generic Graph Metric Estimation. In *ICDE*. 1922–1925.
[67] Shuzhan Ye, Lu Chen, Zhikun Zhang, Yunjun Gao, Yuxiang Wang, and Xiaoliang Xu. 2025. PrivAGS: Differentially Private Attributed Graph Synthesis. *Proceedings of the ACM on Management of Data* 3, 6 (2025).
[68] Feiyu Yin, Yong Liu, Zhiqi Shen, Lisi Chen, Shuo Shang, and Peng Han. 2023. Next POI Recommendation with Dynamic Graph and Explicit Dependency. In *AAAI*. 4827–4834.
[69] Quan Yuan, Zhikun Zhang, Linkang Du, Min Chen, Peng Cheng, and Mingyang Sun. 2023. PrivGraph: Differentially Private Graph Data Publication by Exploiting Community Information. In *USENIX Security Symposium*.
[70] Quan Yuan, Zhikun Zhang, Linkang Du, Min Chen, Mingyang Sun, Yunjun Gao, Michael Backes, Shibo He, and Jiming Chen. 2024. PSGraph: Differentially Private Streaming Graph Synthesis by Considering Temporal Dynamics. *CoRR abs/2412.11369* (2024).
[71] Zygmunt Zajac. 2017. Goodbooks-10k: A New Dataset for Book Recommendations. http://fastml.com/goodbooks-10k. *FastML* (2017).
[72] Jun Zhang, Graham Cormode, Cecilia M Procopiuc, Divesh Srivastava, and Xiaokui Xiao. 2015. Private Release of Graph Statistics Using Ladder Functions. In *SGIMOD*. 731–745.
[73] Jun Zhang, Graham Cormode, Cecilia M Procopiuc, Divesh Srivastava, and Xiaokui Xiao. 2017. PrivBayes: Private Data Release via Bayesian Networks. *ACM Transactions on Database Systems (TODS)* 42, 4 (2017), 1–41.
[74] Sen Zhang, Haibo Hu, Qingqing Ye, and Jianliang Xu. 2025. PrivDPR: Synthetic Graph Publishing with Deep PageRank under Differential Privacy. In *KDD*. 1936–1947.
[75] Xinyang Zhang, Shouling Ji, and Ting Wang. 2018. Differentially Private Releasing via Deep Generative Model. *CoRR* abs/1801.01594 (2018).
[76] Xianhang Zhang, Hanchen Wang, Jianke Yu, Chen Chen, Xiaoyang Wang, and Wenjie Zhang. 2023. Bipartite Graph Capsule Network. In *WWW*.
[77] Zhikun Zhang, Tianhao Wang, Jean Honorio, Ninghui Li, Michael Backes, Shibo He, Jiming Chen, and Yang Zhang. 2021. PrivSyn: Differentially Private Data Synthesis. In *USENIX Security Symposium*.
[78] Tao Zhou. 2021. Progresses and Challenges in Link Prediction. *iScience* (2021).
[79] Tao Zhou, Yan-Li Lee, and Guannan Wang. 2021. Experimental Analyses on 2-hop-based and 3-hop-based Link Prediction Algorithms. *Physica A: Statistical Mechanics and its Applications* (2021).
[80] Xiaochen Zhu, Vincent Y. F. Tan, and Xiaokui Xiao. 2023. Blink: Link Local Differential Privacy in Graph Neural Networks via Bayesian Estimation. In *CCS*. 2651–2664.

A ALGORITHM ANALYSIS

A.1 Privacy Analysis

Theorem 5. JBGS *satisfies ε-edge DP.*

Proof. Initially, the privacy budget ε_e is used to perturb the edge count. The remaining budget $\varepsilon_r = \varepsilon - \varepsilon_e$ is used for subsequent stages. The partitioning decisions are made using the perturbed edge count or previously released perturbed degree information. Therefore, the judgment function in Equation 4 and the resulting partitioning decisions are post-processing of previously released differentially private information and incur no additional privacy loss.

Next, we analyze the budget allocation of ε_r in three cases. In the first case (*i.e.*, no block partitioning), JBGS uses ε_r to perturb the global degree sequences, as in DBGS. In the second case (*i.e.*, a single partitioning operation), JBGS uses $r_1 \cdot \varepsilon_r$ to perturb the global degree sequences and $r_2 \cdot \varepsilon_r$ to perturb the regional degree sequences in each block, as in BKGS.

In the third case (multiple partitioning operations), the global degree sequences are perturbed using $r_1 \cdot \varepsilon_r$. Suppose that the graph undergoes $|J|$ partitioning operations. At the j-th partitioning operation, the current recursively processed block is divided into four blocks, denoted by $S_1^j, S_2^j, S_3^j, S_4^j$. The blocks S_2^j, S_3^j, S_4^j are processed at the current level, whereas S_1^j is the only block eligible for further recursive partitioning. The regional degree information for S_2^j, S_3^j, S_4^j is perturbed using privacy budget $r_2^j \cdot \varepsilon_r$ in each block; parallel composition bounds the joint privacy loss across these disjoint blocks by $r_2^j \cdot \varepsilon_r$. The remaining budget available for S_1^j is also $r_2^j \cdot \varepsilon_r$.

For each nonterminal partitioning operation, where $j < |J|$, the budget assigned to S_1^j is divided into $r_1 r_2^j \cdot \varepsilon_r$ for global degree perturbation and $r_2^{j+1} \cdot \varepsilon_r$ for further regional processing. After the final partitioning operation, where $j = |J|$, no further partitioning is performed, and the entire remaining budget $r_2^{|J|} \cdot \varepsilon_r$ is used for degree perturbation within the terminal high-degree block.

To bound the privacy loss, consider any edge in the original graph. At each nonterminal level, the edge either belongs to one of the blocks processed at that level or remains in the recursively partitioned block S_1^j. If it belongs to a block processed at level j, the privacy budget charged along its path is at most

$$r_1 \cdot \varepsilon_r + \sum_{i=1}^{j-1} r_1 r_2^i \cdot \varepsilon_r + r_2^j \cdot \varepsilon_r = \varepsilon_r, \tag{13}$$

where the first term accounts for the root-level global perturbation, the summation accounts for subsequent global perturbations along the recursive path, and the final term accounts for the regional perturbation at level j. If the edge remains in the terminal block, its privacy loss is bounded by

$$r_1 \cdot \varepsilon_r + \sum_{i=1}^{|J|-1} r_1 r_2^i \cdot \varepsilon_r + r_2^{|J|} \cdot \varepsilon_r = \varepsilon_r, \tag{14}$$

where the equality in both expressions follows from $r_1 + r_2 = 1$.

Thus, under the stated recursive budget allocation and disjoint-block structure, and conditional on previous private outputs at each adaptive step, the privacy loss from all degree perturbation stages

is bounded by ε_r. Including the initial edge-count perturbation, the total privacy loss is at most $\varepsilon_e + \varepsilon_r = \varepsilon$.

Finally, the consistency processing and edge generation stages operate exclusively on the perturbed information. By the post-processing property of DP, they incur no additional privacy loss. Therefore, JBGS satisfies ε-edge DP. □

A.2 Utility Analysis

In this part, we analyze the error introduced by the vertex degree perturbation.

Theorem 6 (Utility Bounds for Degree Perturbation). *Let $G = (U, V, E)$ be a bipartite graph, where $n_U = |U|$ and $n_V = |V|$. For each $u \in U$, let d_u denote its true degree. The perturbed degree is defined as $\hat{d}_u = \max\{0, \text{round}(d_u + Z_u)\}$, where $Z_u \sim Lap(0, 1/\varepsilon_U)$. Similarly, for each $v \in V$, $\hat{d}_v = \max\{0, \text{round}(d_v + Z_v)\}$, where $Z_v \sim Lap(0, 1/\varepsilon_V)$. Then the following bounds hold.*

(i) Expected error for an individual vertex. For every $u \in U$ and $v \in V$,

$$\mathbb{E}\left[|\hat{d}_u - d_u|\right] \le \frac{1}{2\sinh(\varepsilon_U/2)}, \tag{15}$$

$$\mathbb{E}\left[|\hat{d}_v - d_v|\right] \le \frac{1}{2\sinh(\varepsilon_V/2)}. \tag{16}$$

(ii) Expected ℓ_1 error of the degree sequences. Let

$$E_U = \sum_{u\in U} |\hat{d}_u - d_u|, \qquad E_V = \sum_{v\in V} |\hat{d}_v - d_v|. \tag{17}$$

Then

$$\mathbb{E}[E_U] \le \frac{n_U}{2\sinh(\varepsilon_U/2)}, \tag{18}$$

$$\mathbb{E}[E_V] \le \frac{n_V}{2\sinh(\varepsilon_V/2)}. \tag{19}$$

Consequently, the total degree error satisfies

$$\mathbb{E}[E_U + E_V] \le \frac{n_U}{2\sinh(\varepsilon_U/2)} + \frac{n_V}{2\sinh(\varepsilon_V/2)}. \tag{20}$$

(iii) High-probability maximum error bound. For any $\delta \in (0, 1)$, with probability at least $1 - \delta$, the following inequalities hold simultaneously:

$$\max_{u\in U} |\hat{d}_u - d_u| \le \frac{\ln(2n_U/\delta)}{\varepsilon_U} + \frac{1}{2}, \tag{21}$$

$$\max_{v\in V} |\hat{d}_v - d_v| \le \frac{\ln(2n_V/\delta)}{\varepsilon_V} + \frac{1}{2}. \tag{22}$$

Proof. We first establish a bound for an arbitrary vertex whose true degree is a nonnegative integer d. Let $Z \sim Lap(0, 1/\varepsilon), W = \text{round}(Z)$. Since d is an integer, rounding to the nearest integer gives the following equality almost surely:

$$\text{round}(d + Z) = d + W. \tag{23}$$

Thus, the released degree can be written as

$$\hat{d} = \max\{0, d + W\}. \tag{24}$$

Because $d \ge 0$, truncation at zero cannot increase the absolute error. More precisely,

$$|\hat{d} - d| = |\max\{0, d + W\} - d| \le |W|. \tag{25}$$

For every integer $k \ge 1$, the probability that the rounded noise equals k is

$$\begin{aligned} \Pr[W = k] &= \Pr\left[k - \tfrac{1}{2} \le Z < k + \tfrac{1}{2}\right] \\ &= \frac{1}{2}\left(e^{-\varepsilon(k-\frac{1}{2})} - e^{-\varepsilon(k+\frac{1}{2})}\right) \\ &= e^{-\varepsilon k}\sinh(\varepsilon/2). \end{aligned} \tag{26}$$

By the symmetry of Laplace distribution, $\Pr[W = -k] = \Pr[W = k]$. Therefore,

$$\mathbb{E}[|W|] = 2\sum_{k=1}^{\infty} k\Pr[W = k] = 2\sinh(\varepsilon/2)\sum_{k=1}^{\infty} k e^{-\varepsilon k}. \tag{27}$$

Using the identity

$$\sum_{k=1}^{\infty} k q^k = \frac{q}{(1-q)^2}, \qquad |q| < 1, \tag{28}$$

and setting $q = e^{-\varepsilon}$, we obtain

$$\mathbb{E}[|W|] = \frac{2\sinh(\varepsilon/2)e^{-\varepsilon}}{(1-e^{-\varepsilon})^2} = \frac{1}{2\sinh(\varepsilon/2)}. \tag{29}$$

Combining this identity with Equation 25 yields

$$\mathbb{E}[|\hat{d} - d|] \le \frac{1}{2\sinh(\varepsilon/2)}. \tag{30}$$

Applying Equation 30 with $\varepsilon = \varepsilon_U$ to every vertex in U and summing the resulting inequalities gives

$$\mathbb{E}[E_U] = \sum_{u\in U} \mathbb{E}[|\hat{d}_u - d_u|] \le \frac{n_U}{2\sinh(\varepsilon_U/2)}. \tag{31}$$

The same argument gives the corresponding bound for E_V. Adding the two bounds proves Equation 20.

For the high-probability bound, Equation 25 and the rounding-error bound $|\text{round}(Z)| \le |Z| + \frac{1}{2}$ give

$$|\hat{d} - d| \le |Z| + \frac{1}{2}. \tag{32}$$

The Laplace tail probability satisfies

$$\Pr[|Z| > t] = e^{-\varepsilon t}, \qquad t \ge 0. \tag{33}$$

Consequently, for any $t \ge 0$,

$$\Pr\left[|\hat{d} - d| > t + \frac{1}{2}\right] \le e^{-\varepsilon t}. \tag{34}$$

Applying the union bound to the n_U vertices in U and setting

$$t_U = \frac{\ln(2n_U/\delta)}{\varepsilon_U} \tag{35}$$

gives

$$\Pr\left[\max_{u\in U} |\hat{d}_u - d_u| > \frac{\ln(2n_U/\delta)}{\varepsilon_U} + \frac{1}{2}\right] \le \frac{\delta}{2}. \tag{36}$$

Analogously, the probability that the corresponding bound for V fails is at most $\delta/2$. A final application of the union bound shows that both bounds hold simultaneously with probability at least $1 - \delta$. This completes the proof. □

A.3 Computational Cost

Here, we analyze the computational cost of our proposed methods. Let $n_U = |U|$ and $n_V = |V|$ denote the number of upper and lower vertices, respectively, and let $N = n_U + n_V$. Let m denotes the number of edges in the synthetic graph. D is the maximum recursive partitioning depth ($D \geq 1$), and R is the maximum number of edge-correction rounds during degree restoration.

For DBGS, perturbing the degree sequences of all vertices in U and V requires $O(N)$ time. The initial synthetic graph is then constructed according to the perturbed upper-vertex degrees, requiring $O(N + m)$ time. Finally, the edge-correction procedure adjusts the edges to reduce discrepancies from the perturbed lower-vertex degree targets. Since each correction round takes $O(m)$ time and at most R rounds are performed, this step requires $O(Rm)$ time. Therefore, the overall computational cost of DBGS is $O(N + Rm)$.

For BKGS, perturbing the global degree sequences requires $O(N)$ time. The bipartite graph is then divided into four blocks based on the perturbed degree information, requiring $O(N)$ time. Assigning the original edges to their corresponding blocks takes $O(m)$ time. Although vertices occur in more than one block, each occurs in at most two of the four blocks. Then, perturbing the regional degree sequences across all blocks requires $O(N)$ time, while reconstructing the blocks requires $O(Rm)$ time in total. Thus, the overall computational cost of BKGS is $O(N + Rm)$.

For JBGS, the graph may be recursively partitioned until the stopping criterion is met. At each level, the total number of vertices and edges across all active blocks is bounded by N and m, respectively. Consequently, degree processing and edge assignment require $O(N + m)$ time per level, while partitioning the vertices across the active blocks takes at most $O(N)$ time per level. Therefore, the recursive partitioning procedure requires $O(D(N + m))$ time in total. After partitioning, redistributing the perturbed degrees among the leaf blocks requires $O(N)$ time, and reconstructing all leaf blocks requires $O(Rm)$ time in total. Thus, the overall computational cost is $O\left(D(N + m) + Rm\right)$.